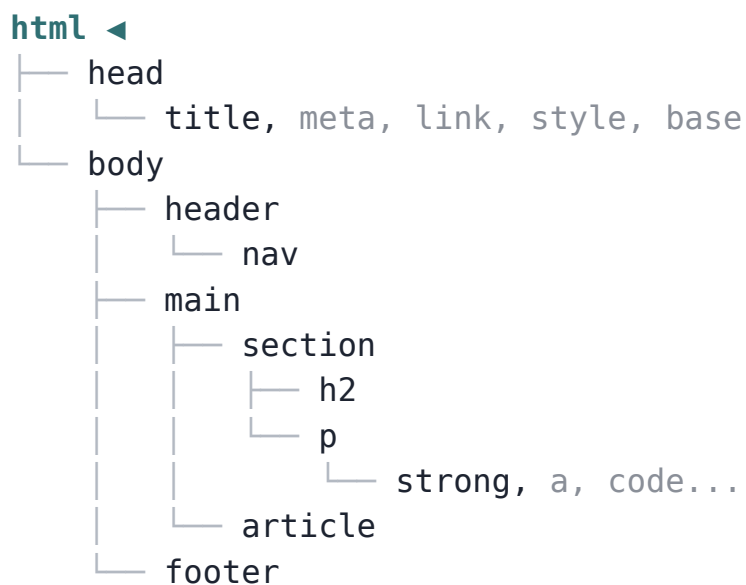


The HTML Tree

Every HTML element, where it grows in the DOM tree, and two structural exercises for each — 114 elements, 218 exercises.



A page is not a list of tags. It is a tree: one root, branches, and leaves. Learn where each element is allowed to grow and you can read — and write — any page.

How the tree works

When the browser reads HTML it builds the **DOM** (Document Object Model): a tree of nodes. Every tag you write becomes a node. Five words describe every relationship in that tree:

Root	The single node with no parent: <code><html></code> .
Parent	The element directly containing another. In <code></code> , <code></code> is the parent of <code></code> .
Child	An element directly inside another. <code></code> is a child of <code></code> .
Siblings	Children of the same parent. <code><head></code> and <code><body></code> are siblings.
Depth	How many steps from the root. <code>html</code> = 0, <code>body</code> = 1, <code>main</code> = 2 ...

The three rules of growth

1. Every element has exactly one parent (except the root). That's why tags must close in reverse order: `<p><strong>ok</strong></p>`, never `<p><strong>bad</p></strong>`.

2. Each element only accepts certain children. `<ul>` only accepts `<li>`; `<p>` only accepts inline content. Break the rule and the browser silently rebuilds the tree its own way.

3. Meaning flows down. A `<footer>` inside an `<article>` is that article's footer, not the page's. Where an element sits changes what it means.

How to read each card

Parent tells you where the element is allowed to live. **Children** tells you what may grow inside it. The grey tree shows the element in a typical context; the coloured name marked ◀ is the one being studied. **VOID** elements have no children and no closing tag.

`<li>`

example

Parent `<ul>`, `<ol>`, `<menu>` | **Children** flow content

What the element is for, in one or two lines.

WHERE IT LIVES IN THE TREE

```
ol
├── li ◀
│   └── p
└── li
```

EXERCISE 1

The first exercise builds the element into a small working structure.

EXERCISE 2

The second pushes further: nesting it deeper, combining it with siblings, or breaking a rule on purpose to see what the browser does.

How to use the exercises

For every exercise: **(1)** draw the tree on paper first, **(2)** write the HTML, **(3)** open DevTools → Elements and compare the browser's tree with your drawing, **(4)** run it through validator.w3.org. If the drawing and the DevTools tree differ, you've found a rule — write it down.

The whole tree on one page

Every branch of this book, in the order the DOM grows: from the root, into the head, then down through the body.

1	The Root and the Head html > head	html, head, title, meta, link, style, base
2	Body and Page Landmarks html > body > landmarks	body, header, nav, main, section, article, aside, footer, address, h1 – h6, hgroup, search
3	Grouping Content html > body > ... > blocks	div, p, hr, pre, blockquote, ol, ul, menu, li, dl, dt, dd, figure, figcaption
4	Text-Level Elements html > body > ... > p > inline	a, em, strong, small, s, cite, q, dfn, abbr, ruby, rt, rp, data, time, code, var, samp, kbd, sub, sup, i, b, u, mark, bdi, bdo, span, br, wbr
5	Edits html > body > ... > changes	ins, del
6	Embedded Content html > body > ... > media	picture, source, img, iframe, embed, object, video, audio, track, map, area, svg, math
7	Tables html > body > ... > table	table, caption, colgroup, col, thead, tbody, tfoot, tr, th, td
8	Forms html > body > ... > form	form, label, input, button, select, datalist, optgroup, option, textarea, output, progress, meter, fieldset, legend
9	Interactive Elements html > body > ... > widgets	details, summary, dialog
10	Scripting and Components html > head/body > scripts	script, noscript, template, slot, canvas

The shape to remember

html ◀

```
├─ head (branch 1)
├─ body (branch 2)
│   ├── landmarks: header, nav, main, aside, footer
│   │   └─ sections: section, article, h1–h6
│   │       ├── blocks: p, ul, ol, dl, figure, pre (branch 3)
│   │       │   └─ inline: a, strong, em, code... (branch 4, 5)
│   │       ├── media: img, video, iframe, svg (branch 6)
│   │       ├── table > tr > td (branch 7)
│   │       ├── form > fieldset > input (branch 8)
│   │       └─ details, dialog (branch 9)
│   └─ script, template, canvas (branch 10)
```

The Root and the Head

html > head

Every document is one tree with a single root: `<html>`. The root has exactly two children, `<head>` (information about the page, invisible) and `<body>` (the page itself, visible). This branch covers the root and everything that grows under `<head>`.

<html>

001

Parent none — it is the root | **Children** one `<head>`, then one `<body>`

The root node of the tree. Every other element is its descendant. Set `lang` here so browsers, translators and screen readers know the page language.

WHERE IT LIVES IN THE TREE

```
html lang="en" ◀
├── head
│   └── title
└── body
```

EXERCISE 1

Write the minimum valid document: `<!DOCTYPE html>`, `<html lang="en">`, a `<head>` with a `<title>`, and an empty `<body>`. Draw the tree on paper first, then type it.

EXERCISE 2

Open any website, open DevTools (F12) → Elements, and collapse everything. Confirm the root has exactly two children. Write down their names and the `lang` value.

<head>

002

Parent `<html>` (first child) | **Children** metadata only: `<title>`, `<meta>`, `<link>`, `<style>`, `<script>`, `<base>`

Container for information about the page. Nothing in it is displayed as page content.

WHERE IT LIVES IN THE TREE

```
html
├── head ◀
│   ├── meta charset
│   ├── title
│   └── link
└── body
```

EXERCISE 1

Put a `<p>Hello</p>` inside `<head>`, reload, and inspect the tree in DevTools. Where did the browser move it? Write one sentence explaining why.

EXERCISE 2

Write a `<head>` containing, in order: charset `<meta>`, viewport `<meta>`, `<title>`, and a stylesheet `<link>`. Draw it as a tree with four children.

<title>

003

Parent `<head>` | **Children** text only

The page's name — shown in the browser tab, bookmarks, and search results. Exactly one per page.

WHERE IT LIVES IN THE TREE

```
html
├── head
│   └── title ◀
│       └── "About – My Site"
└── body
```

EXERCISE 1

Create three pages (Home, About, Contact), each with a different `<title>`. Open all three in tabs and compare.

EXERCISE 2

For a blog post page, write a `<title>` using the pattern `Post Name – Site Name`, and an `<h1>` in the `<body>` with just the post name. Explain in one line why the two differ.

<meta> VOID · no closing tag

004

Parent <head> | **Children** nothing (void)

Metadata that has no element of its own: character set, viewport, description, social-share info.

WHERE IT LIVES IN THE TREE

```
html
├── head
│   ├── meta charset="UTF-8" ◀
│   ├── meta name="viewport" ◀
│   ├── meta name="description" ◀
│   └── title
```

EXERCISE 1

Add charset, viewport and description <meta> tags. Then delete the viewport one and view the page in DevTools' phone mode. Describe the difference.

EXERCISE 2

Add Open Graph tags (og:title, og:description, og:image) to a page. For each one, write what it controls when the link is shared in WhatsApp.

<link> VOID · no closing tag

005

Parent <head> | **Children** nothing (void)

Connects the page to external resources: stylesheets, icons, fonts, preloads.

WHERE IT LIVES IN THE TREE

```
html
├── head
│   ├── title
│   ├── link rel="icon" ◀
│   └── link rel="stylesheet" ◀
```

EXERCISE 1

Link two stylesheets, base.css and theme.css, that both set p { color } differently, plus a favicon. Swap the order of the two <link> tags and note which colour wins.

EXERCISE 2

Add a preconnect <link> and a Google Fonts stylesheet <link>, then use the font on your <h1>.

<style>

006

Parent <head> | **Children** CSS text

Embeds CSS directly in the page.

WHERE IT LIVES IN THE TREE

```
html
├── head
│   ├── title
│   └── style ◀
│       └── "h1 { color: teal }"
└── body
```

EXERCISE 1

Style h1, p and .note with a <style> block. Then move the exact same rules into style.css and connect it with <link>. The page must look identical.

EXERCISE 2

Have a <style> block and an external stylesheet both style <p> differently. Figure out which wins, then reverse it by only changing their order in the tree.

<base> VOID · no closing tag

007

Parent <head> (at most one) | **Children** nothing (void)

Sets the base URL and/or default link target for every relative link on the page.

WHERE IT LIVES IN THE TREE

```
html
├── head
│   ├── base href="https://site.com/" ◀
│   └── title
└── body
    └── a href="about"
```

EXERCISE 1

Set <base href="https://developer.mozilla.org/"> and add three links with relative hrefs like en-US/. Hover each to see the full address.

EXERCISE 2

Add <base target="_blank"> and four links. Confirm all open in new tabs, then override one with target="_self".

Body and Page Landmarks

html > body > landmarks

`<body>` is the second child of `<html>`. Its first level of children are usually landmarks: large named regions (header, nav, main, footer) that give the page its skeleton. Headings then name each region.

`<body>`

008

Parent `<html>` (second child) | **Children** all visible content

Everything the user sees lives here.

WHERE IT LIVES IN THE TREE

```
html
├── head
└── body ◀
    ├── header
    ├── main
    └── footer
```

EXERCISE 1

Draw a tree of `body > header, main, footer`, then write it. Put one sentence of text in each so you can see all three.

EXERCISE 2

Pick a news site you use. Using DevTools, write down the direct children of its `<body>` as a tree. Circle any that should have been landmarks but are `<div>`s.

`<header>`

009

Parent `<body>`, `<article>`, `<section>`, `<main>` | **Children** headings, logo, `<nav>`, intro content

Introductory content for the page or for its nearest section. Cannot sit inside a `<footer>` or another `<header>`.

WHERE IT LIVES IN THE TREE

```
body
├── header ◀
│   ├── img (logo)
│   ├── h1
│   └── nav
└── main
```

EXERCISE 1

Build a site header: logo `<img>`, site name in `<h1>`, and a `<nav>` with three links.

EXERCISE 2

Add an `<article>` below it with its own `<header>` containing an `<h2>` and a `<time>`. Draw the tree and show that the two headers are at different depths.

`<nav>`

010

Parent `<header>`, `<body>`, `<aside>` | **Children** usually `<ul>` > `<li>` > `<a>`

A block of major navigation links. Not every group of links needs one — only primary navigation.

WHERE IT LIVES IN THE TREE

```
header
└── nav ◀
    └── ul
        ├── li
        │   └── a
        └── li
            └── a
```

EXERCISE 1

Build `nav > ul > li > a` for four pages. On the current page's link add `aria-current="page"`.

EXERCISE 2

Make a page with two `<nav>`s: the main menu in the `<header>`, and a table of contents inside an `<aside>` that links to `#ids` of sections on the same page.

<main>

011

Parent <body> | **Children** the page's unique content

The dominant content of the page. Only one visible <main> per page. Never inside <header>, <footer>, <nav>, <aside> or <article>.

WHERE IT LIVES IN THE TREE

```
body
├── header
├── main ◀
│   ├── section
│   └── section
└── footer
```

EXERCISE 1

Refactor a page that uses <div id="content"> to use <main>. Check with DevTools that exactly one <main> exists.

EXERCISE 2

Build `body > header, main, footer` where <main> holds two sections. Draw the tree and label every node with its depth (html = 0).

<section>

012

Parent <main>, <article>, <section> | **Children** a heading + related content

A thematic group of content. It should almost always start with a heading.

WHERE IT LIVES IN THE TREE

```
main
├── section ◀
│   ├── h2 "Services"
│   └── p
└── section ◀
    └── h2 "Pricing"
```

EXERCISE 1

Inside <main>, create three sections (Services, Pricing, Contact), each starting with an <h2>.

EXERCISE 2

Nest sections: a "Courses" section with two sub-sections, "HTML" and "CSS", each with an <h3>. Verify heading level matches nesting depth.

<article>

013

Parent <main>, <section>, <article> | **Children** header, headings, paragraphs, footer

Self-contained content that would still make sense on its own: a post, a product card, a comment.

WHERE IT LIVES IN THE TREE

```
main
├── article ◀
│   ├── header
│   │   └── h2
│   │   └── time
│   ├── p
│   └── footer
```

EXERCISE 1

Build a blog index: <main> with three <article>s, each with a <header> (<h2>, <time>), an excerpt <p>, and a "Read more" link.

EXERCISE 2

Build one full post as an <article> that contains a <section> titled "Comments" holding two nested <article>s (one per comment).

<aside>

014

Parent <body>, <main>, <article> | **Children** any flow content

Content tangentially related to what is around it: sidebars, pull-quotes, "related" boxes.

WHERE IT LIVES IN THE TREE

```
main
├── article
│   ├── p
│   ├── aside ◀
│   │   └── p "Did you know?"
│   └── aside ◀
│       └── h2 "Related"
```

EXERCISE 1

Lay out <main> with an <article> and a sibling <aside> holding a "Related posts" list.

EXERCISE 2

Inside an article, add an <aside> "Did you know?" box next to the paragraph it relates to. Draw both asides in one tree and note how their parents differ.

<footer>

015

Parent <body>, <article>, <section> | **Children** copyright, links, contact, author info

Closing content for the page or for its nearest section. Cannot contain <header> or another <footer>.

WHERE IT LIVES IN THE TREE

```
body
├── main
│   ├── article
│   │   └── footer ◀
│   └── footer ◀
│       └── small
```

EXERCISE 1

Build a site footer with a <small> copyright line, a <nav> of legal links, and an <address>.

EXERCISE 2

Add a <footer> to an article with author name and a list of tags. Now the page has two footers — draw the tree to show why that is valid.

<address>

016

Parent <footer>, <article>, <body> | **Children** contact info (no headings, no sections)

Contact information for the person or organisation responsible for the nearest article or the whole page.

WHERE IT LIVES IN THE TREE

```
footer
└── address ◀
    ├── a href="mailto:"
    ├── br
    └── a href="tel:"
```

EXERCISE 1

In the page footer, write an <address> with an email (mailto:) link and a phone (tel:) link.

EXERCISE 2

Add an <address> inside an article's footer for its author. Then write one sentence on why a random shop's street address in a blog post should NOT use <address>.

<h1> – <h6>

017

Parent any section-like element | **Children** text and inline elements

Six levels of headings (6 elements). They build the page outline. Use one <h1>; never skip levels going down.

WHERE IT LIVES IN THE TREE

```
main
├── h1 "Web Basics" ◀
└── section
    ├── h2 "HTML" ◀
    └── section
        └── h3 "Elements" ◀
```

EXERCISE 1

Write a chapter outline using <h1> to <h4> with no skipped levels. Then redraw that outline as a tree — it should match your element tree.

EXERCISE 2

Given headings in the order h1, h4, h2, h3, fix the document. Install a headings-outline browser extension and compare before and after.

<hgroup>

018

Parent any section-like element | **Children** one heading + <p> elements

Groups a heading with a subtitle or tagline so the tagline isn't mistaken for a heading.

WHERE IT LIVES IN THE TREE

```
header
├── hgroup ◀
│   ├── h1 "Code Club"
│   └── p "Learn to build the web"
└── nav
```

EXERCISE 1

Make a site <header> with an <hgroup> holding an <h1> name and a <p> tagline.

EXERCISE 2

Put an <hgroup> inside an article's header: <h2> title plus a <p> subtitle. Explain why the subtitle is a <p> and not an <h3>.

<search>

Parent <header>, <main>, <aside> | **Children** a search or filter form

Marks the part of the page used for searching or filtering.

WHERE IT LIVES IN THE TREE

```
header
├── nav
└── search ◀
    ├── form
    │   ├── label
    │   ├── input type="search"
    │   └── button
```

EXERCISE 1

Wrap a search form (<label>, <input type="search">, <button>) in <search> inside the <header>.

EXERCISE 2

Build a product filter panel: <search> > <form> with a text input and a category <select>, placed in an <aside>.

Grouping Content

html > body > ... > blocks

Inside landmarks, content is grouped into blocks: paragraphs, lists, quotes, figures. These are the branches that carry most of a page's text.

<div>

020

Parent anywhere flow content is allowed | **Children** any flow content

A generic block container with no meaning. Use it for layout and styling only when no semantic element fits.

WHERE IT LIVES IN THE TREE

```
main
├── section
│   ├── h2
│   └── div class="grid" ◀
│       ├── article
│       ├── article
│       └── article
```

EXERCISE 1

Wrap three article cards in `<div class="grid">` for layout. Write one sentence on why `<div>` (not `<section>`) is right here.

EXERCISE 2

Take a page built entirely from `<div>`s (write one with 10). Replace every div that has a semantic equivalent. Count how many remain.

<p>

021

Parent flow containers | **Children** text and inline elements only

A paragraph. It can only contain inline (phrasing) content — no lists, headings or other blocks.

WHERE IT LIVES IN THE TREE

```
section
├── h2
└── p ◀
    ├── "I build "
    ├── strong "websites"
    └── a "see my work"
```

EXERCISE 1

Write three paragraphs about yourself. One must contain a `<strong>` and a link.

EXERCISE 2

Put a `<ul>` inside a `<p>`, then inspect the result in DevTools. Draw the tree the browser actually built and explain what happened.

<hr> VOID · no closing tag

022

Parent flow containers | **Children** nothing (void)

A thematic break between paragraph-level blocks — a change of scene or topic.

WHERE IT LIVES IN THE TREE

```
article
├── p "Scene one..."
├── hr ◀
└── p "Scene two..."
```

EXERCISE 1

Write a two-scene short story in an `<article>` and separate the scenes with `<hr>`.

EXERCISE 2

Build one divider with `<hr>` and one with a CSS `border-bottom`. Write when each is correct (meaning vs decoration).

<pre>

023

Parent flow containers | **Children** text, often `<code>`

Preformatted text: whitespace and line breaks are shown exactly as typed.

WHERE IT LIVES IN THE TREE

```
figure
├── pre ◀
│   └── code
│       └── "<p>Hi</p>"
```

EXERCISE 1

Draw your page's DOM tree as ASCII art inside a `<pre>`, using spaces for indentation.

EXERCISE 2

Show a five-line HTML snippet on a page using `<pre><code>`. Escape every `<` as `<`; so it displays instead of rendering.

<blockquote>

024

Parent flow containers | **Children** flow content (paragraphs etc.)

A longer quotation from another source. Use `cite` for the source URL.

WHERE IT LIVES IN THE TREE

```
figure
├── blockquote cite="url" ◀
│   ├── p
│   ├── p
│   └── figcaption
│       └── cite
```

EXERCISE 1

Build `figure > blockquote + figcaption` with a quote and its author.

EXERCISE 2

Write a `<blockquote>` with a `cite` URL containing two paragraphs, placed inside an `<article>`.

025

Parent flow containers | **Children** `<li>` only

An ordered (numbered) list — order matters. Supports `start`, `reversed`, `type`.

WHERE IT LIVES IN THE TREE

```
article
├── h2 "Recipe"
└── ol ◀
    ├── li
    ├── li
    └── ol ◀
        └── li
```

EXERCISE 1

Write recipe steps in an `<ol>`. Then make a countdown list using `reversed`, and one starting at 5 using `start`.

EXERCISE 2

Write a nested course outline: `ol > li > ol` for chapters and their sub-sections. Draw it as a tree.

026

Parent flow containers | **Children** `<li>` only

An unordered list — order doesn't matter.

WHERE IT LIVES IN THE TREE

```
nav
├── ul ◀
│   ├── li
│   │   └── a
│   ├── li
│   │   └── a
│   └── ul ◀
│       └── li
```

EXERCISE 1

Write a shopping list with a nested sub-list under "Fruit".

EXERCISE 2

Build a menu: `nav > ul > li > a`, where one item has a nested `<ul>` submenu.

<menu>

027

Parent flow containers | **Children** only

A semantic alternative to for a list of commands or actions (a toolbar).

WHERE IT LIVES IN THE TREE

```
div class="editor"
├── menu ◀
│   ├── li
│   │   └── button "Bold"
│   └── li
│       └── button "Italic"
```

EXERCISE 1

Build an editor toolbar: `menu > li > button` for Bold, Italic, Underline.

EXERCISE 2

Side by side, build a list of links () and a list of actions (<menu>). Write one line on the difference.

028

Parent , , <menu> | **Children** flow content

A list item. Its parent must be a list. It can contain blocks, even other lists.

WHERE IT LIVES IN THE TREE

```
ol
├── li ◀
│   ├── p
│   └── img
└── li value="10" ◀
```

EXERCISE 1

Write an where each contains an <h3>, a <p>, and an — proof that list items can hold blocks.

EXERCISE 2

Use the `value` attribute on to make a numbered list that skips from 3 to 10.

<dl>

029

Parent flow containers | **Children** <dt> + <dd> groups (optionally wrapped in <div>)

A description list: name–value groups like a glossary or spec sheet.

WHERE IT LIVES IN THE TREE

```
section
├── dl ◀
│   ├── div
│   │   ├── dt
│   │   └── dd
│   └── div
│       ├── dt
│       └── dd
```

EXERCISE 1

Write a glossary of five HTML terms using `dl > dt + dd`.

EXERCISE 2

Write a phone spec sheet (screen, battery, storage) as a <dl> where each pair is wrapped in a <div>.

<dt>

030

Parent <dl> (or <div> inside it) | **Children** phrasing content

The term or name in a description list.

WHERE IT LIVES IN THE TREE

```
dl
├── dt "img" ◀
├── dt "image" ◀
└── dd "Embeds a picture"
```

EXERCISE 1

Write one <dt> with two <dd>s (a word with two meanings).

EXERCISE 2

Write two <dt>s sharing one <dd> (synonyms).

<dd>

031

Parent <dl> (or <div> inside it) | **Children** flow content

The description or value for the preceding term(s).

WHERE IT LIVES IN THE TREE

```
dl
├── dt "What is the DOM?"
└── dd ◀
    ├── p
    └── ul
```

EXERCISE 1

Write a <dd> containing a <p> and a of examples.

EXERCISE 2

Build an FAQ as a <dl>: questions in <dt>, answers in <dd> with at least one link.

<figure>

032

Parent flow containers | **Children** content + optional <figcaption>

Self-contained content (image, code, chart, quote) that is referenced from the main text.

WHERE IT LIVES IN THE TREE

```
article
├── p
└── figure ◀
    ├── img
    └── figcaption
```

EXERCISE 1

Build `figure > img + figcaption` for a photo.

EXERCISE 2

Build a <figure> holding `pre > code` with the caption "Listing 1: A basic page".

<figcaption>

033

Parent <figure> (first or last child) | **Children** flow content

The caption for its parent <figure>.

WHERE IT LIVES IN THE TREE

```
figure
├── img "before"
├── img "after"
└── figcaption ◀
```

EXERCISE 1

Put the <figcaption> first, then move it last. Both are valid — confirm by validating.

EXERCISE 2

Make a before/after <figure> with two images and one caption describing both.

Text-Level Elements

html > body > ... > p > inline

These are the leaves. They live inside paragraphs, headings and list items, and add meaning to words and phrases. Most are inline: they sit inside a line of text instead of starting a new block.

<a>

034

Parent almost anywhere | **Children** text, inline content, or blocks (no nested <a>)

A hyperlink. `href` is the destination.

WHERE IT LIVES IN THE TREE

```
nav
├── ul
│   └── li
│       └── a href="#pricing" ◀
p
└── a href="mailto:" ◀
```

EXERCISE 1

Create four links: external (`target="_blank"` `rel="noopener"`), email (`mailto:`), phone (`tel:`), and a file with `download`.

EXERCISE 2

Build a table of contents: a <nav> of # links that jump to `ids` on three sections of the same page.

035

Parent phrasing content | **Children** phrasing content

Stress emphasis — changes how the sentence is read aloud.

WHERE IT LIVES IN THE TREE

```
p
├── "I "
├── em "never" ◀
└── " said that"
```

EXERCISE 1

Write "I never said she took the money" three times, with on a different word each time. Explain how the meaning changes.

EXERCISE 2

Nest inside for stronger stress in a sentence. Draw the tree.

036

Parent phrasing content | **Children** phrasing content

Strong importance, seriousness or urgency.

WHERE IT LIVES IN THE TREE

```
p
├── strong "Warning:" ◀
└── " do not unplug"
```

EXERCISE 1

Write a safety notice where only the critical part is in .

EXERCISE 2

Write one sentence where is correct and one where is correct. Explain the difference.

<small>

037

Parent phrasing content | **Children** phrasing content

Side comments and fine print: legal text, copyright, caveats.

WHERE IT LIVES IN THE TREE

```
footer
└─ p
   └─ small "© 2026 My Site" ◀
```

EXERCISE 1

Add a `<small>` copyright notice in a footer paragraph.

EXERCISE 2

Show a price with `<small>(incl. VAT)</small>` next to it inside a product card.

<S>

038

Parent phrasing content | **Children** phrasing content

Content that is no longer accurate or relevant (but wasn't an edit).

WHERE IT LIVES IN THE TREE

```
p
└─ s "KES 2,000" ◀
   └─ " KES 1,500"
```

EXERCISE 1

Show a sale price: old price in `<s>`, new price after it.

EXERCISE 2

Build one example with `<s>` and one with `<del>`. Write when each is correct.

<cite>

039

Parent phrasing content | **Children** phrasing content

The title of a creative work: book, film, song, article.

WHERE IT LIVES IN THE TREE

```
figcaption
└─ "– Author, "
   └─ cite "Book Title" ◀
```

EXERCISE 1

Write a paragraph recommending two books, each title in `<cite>`.

EXERCISE 2

Build `figure > blockquote + figcaption`, where the caption contains the author name and the work's title in `<cite>`.

<q>

040

Parent phrasing content | **Children** phrasing content

A short inline quotation. The browser adds quote marks.

WHERE IT LIVES IN THE TREE

```
p
└─ "She said "
   └─ q "ship it" ◀
```

EXERCISE 1

Write a sentence with an inline `<q>`. Notice you don't type the quote marks.

EXERCISE 2

Nest a `<q>` inside another `<q>` and observe how the quote-mark style changes.

<dfn>

041

Parent phrasing content | **Children** phrasing content (no <dfn>)

The defining instance of a term — where it is first explained.

WHERE IT LIVES IN THE TREE

```
p
├── dfn id="dom" ◀
│   └── abbr "DOM"
│       └── " is the tree..."
```

EXERCISE 1

Write a paragraph defining "the DOM" with the term in <dfn>.

EXERCISE 2

Give the <dfn> an **id**, then link to it (**#id**) from a later paragraph where the term is used again.

<abbr>

042

Parent phrasing content | **Children** phrasing content

An abbreviation or acronym. **title** gives the full form.

WHERE IT LIVES IN THE TREE

```
p
├── abbr title="HyperText..." "HTML" ◀
├── " and "
└── abbr "CSS" ◀
```

EXERCISE 1

Write a sentence using HTML, CSS and API, each in <abbr> with a **title** expansion. Hover to test.

EXERCISE 2

Combine **dfn** > **abbr** to define "W3C" the first time it appears.

<ruby>

043

Parent phrasing content | **Children** base text, <rt>, <rp>

Ruby annotation — small text above base text, mainly for pronunciation in East Asian scripts.

WHERE IT LIVES IN THE TREE

```
p
├── ruby ◀
│   ├── "HTML"
│   ├── rp "("
│   ├── rt "H-T-M-L"
│   └── rp ")"
```

EXERCISE 1

Use <ruby> to show the pronunciation above an English word, e.g. "colonel" with "KER-nel".

EXERCISE 2

Use <ruby> to annotate three Swahili words with their English meaning shown above each.

<rt>

044

Parent <ruby> | **Children** phrasing content

The annotation text inside a <ruby>.

WHERE IT LIVES IN THE TREE

```
ruby
├── "cat"
├── rt "noun" ◀
├── "run"
└── rt "verb" ◀
```

EXERCISE 1

Build one <ruby> with two base/annotation pairs.

EXERCISE 2

Style **rt** with CSS to be smaller and coloured. Draw the ruby subtree.

<rp>

045

Parent <ruby> | **Children** text

Fallback parentheses shown only by browsers that don't support ruby.

WHERE IT LIVES IN THE TREE

```
ruby
├── "HTML"
├── rp "(" ◀
├── rt
├── rp ")" ◀
```

EXERCISE 1

Add <rp> around an <rt> and view the page source text to see the fallback.

EXERCISE 2

Write the children of a <ruby> in the correct order: base, rp, rt, rp. Explain the order.

<data>

046

Parent phrasing content | **Children** phrasing content

Links human-readable text to a machine-readable *value*.

WHERE IT LIVES IN THE TREE

```
ul
├── li
│   └── data value="SKU-104" "Blue Shirt" ◀
```

EXERCISE 1

Build a product list where each name is wrapped in <data value="SKU-...">.

EXERCISE 2

Build a scores table where each score cell uses <data value> for sorting.

<time>

047

Parent phrasing content | **Children** phrasing content

A date, time or duration with a machine-readable *datetime*.

WHERE IT LIVES IN THE TREE

```
article
├── header
│   ├── h2
│   └── time datetime="2026-10-05" ◀
```

EXERCISE 1

Announce an event using <time datetime="2026-10-05T18:00">.

EXERCISE 2

In an article header, add a publish date and a reading time using datetime="PT5M".

<code>

048

Parent phrasing content | **Children** phrasing content

A fragment of computer code.

WHERE IT LIVES IN THE TREE

```
p
├── "Use the "
├── code "<p>" ◀
├── " tag"
pre
├── code ◀
```

EXERCISE 1

Write a paragraph explaining three tags, each tag name in <code>.

EXERCISE 2

Show a full code block with pre > code.

<var>

049

Parent phrasing content | **Children** phrasing content

A variable in maths or programming.

WHERE IT LIVES IN THE TREE

```
p
├── var "area" ◀
├── " = "
├── var "w" ◀
├── " x "
└── var "h" ◀
```

EXERCISE 1

Write the formula for a rectangle's area using `<var>`.

EXERCISE 2

Write an instruction: "Set count to 0, then add 1" with `count` in `<var>`.

<samp>

050

Parent phrasing content | **Children** phrasing content

Sample output from a program.

WHERE IT LIVES IN THE TREE

```
pre
├── kbd "npm run dev"
└── samp "Ready on :3000" ◀
```

EXERCISE 1

Show a terminal session in `<pre>`: the command in `<kbd>`, the output in `<samp>`.

EXERCISE 2

Show an error message a user might see, inside `<samp>`, within a help article.

<kbd>

051

Parent phrasing content | **Children** phrasing content

User input, typically keys pressed.

WHERE IT LIVES IN THE TREE

```
p
├── "Press "
└── kbd ◀
    ├── kbd "Ctrl"
    ├── "+"
    └── kbd "S"
```

EXERCISE 1

Write shortcuts for save, copy and paste using `<kbd>`.

EXERCISE 2

Nest `<kbd>` inside `<kbd>` for key combinations. Draw the tree.

<sub>

052

Parent phrasing content | **Children** phrasing content

Subscript — for chemistry, maths indexes.

WHERE IT LIVES IN THE TREE

```
p
├── "H"
├── sub "2" ◀
└── "0"
```

EXERCISE 1

Write H₂O and CO₂ correctly with `<sub>`.

EXERCISE 2

Write a sequence x₁, x₂, ... x_n with `<var>` and `<sub>`.

<sup>

053

Parent phrasing content | **Children** phrasing content

Superscript — exponents, footnote markers.

WHERE IT LIVES IN THE TREE

```
p
├── "E = mc"
├── sup "2" ◀
└── sup ◀
    ├── a href="#fn1" "1"
```

EXERCISE 1

Write $E = mc^2$ and 1st, 2nd, 3rd with <sup>.

EXERCISE 2

Build a footnote system: `sup > a` linking to `#fn1` in a footer list, with a back-link.

<i>

054

Parent phrasing content | **Children** phrasing content

Text in an alternate voice: foreign words, technical terms, thoughts, species names.

WHERE IT LIVES IN THE TREE

```
p
├── i lang="sw" "Karibu" ◀
└── " means welcome"
```

EXERCISE 1

Write a sentence with a Swahili phrase in `<i lang="sw">`.

EXERCISE 2

Write a paragraph about an animal with its scientific name in `<i>`.

055

Parent phrasing content | **Children** phrasing content

Draws attention without adding importance: keywords, product names, lead sentences.

WHERE IT LIVES IN THE TREE

```
article
└── p
    ├── b "Nairobi, Monday." ◀
    └── " The city..."
```

EXERCISE 1

Write a product review with product names in `<b>`.

EXERCISE 2

Put an article's lead sentence in `<b>`. Explain why `<strong>` is wrong here.

<u>

056

Parent phrasing content | **Children** phrasing content

An unarticulated annotation, e.g. marking a spelling error.

WHERE IT LIVES IN THE TREE

```
p
├── "I recieved "
```

```
└── u class="spell" "recieved" ◀
```

EXERCISE 1

Mark two misspelled words with `<u class="spelling">` and style them with a red wavy underline.

EXERCISE 2

Build a paragraph with a link and a `<u>`. Style them so users can't confuse them.

<mark>

057

Parent phrasing content | **Children** phrasing content

Highlighted for relevance, e.g. search matches.

WHERE IT LIVES IN THE TREE

```
li
├── p
│   ├── "Learn "
│   ├── mark "HTML" ◀
│   └── " fast"
```

EXERCISE 1

Show three search results with the search term wrapped in `<mark>`.

EXERCISE 2

Quote a passage and `<mark>` the part you're discussing.

<bdi>

058

Parent phrasing content | **Children** phrasing content

Isolates text whose direction is unknown (e.g. usernames in right-to-left languages).

WHERE IT LIVES IN THE TREE

```
ol
├── li
│   ├── bdi "username" ◀
│   └── ": 90 points"
```

EXERCISE 1

Build a leaderboard where each username is in `<bdi>`.

EXERCISE 2

Build a comment list "[user] wrote: ..." with each user in `<bdi>`.

<bdo>

059

Parent phrasing content | **Children** phrasing content

Overrides text direction with `dir`.

WHERE IT LIVES IN THE TREE

```
p
└── bdo dir="rtl" "Hello" ◀
```

EXERCISE 1

Reverse a word visually with `<bdo dir="rtl">`.

EXERCISE 2

Show the same sentence twice, once `dir="ltr"` and once `dir="rtl"`.

060

Parent phrasing content | **Children** phrasing content

A generic inline container with no meaning — the inline version of `<div>`.

WHERE IT LIVES IN THE TREE

```
p
├── span class="currency" "KES" ◀
└── span class="amount" "1,500" ◀
```

EXERCISE 1

Colour one word in a sentence using a `<span class>`.

EXERCISE 2

Display a price as two spans, currency and amount, and style them differently.

 VOID · no closing tag

061

Parent phrasing content | **Children** nothing (void)

A line break inside text, where the break is part of the content (poems, addresses).

WHERE IT LIVES IN THE TREE

```
address
├── "Moi Avenue"
├── br ◀
└── "Nairobi"
```

EXERCISE 1

Write a postal address in `<address>` using `<br>`.

EXERCISE 2

Write a four-line poem stanza with `<br>`. Then explain why `<br><br>` must not be used for spacing between paragraphs.

<wbr> VOID · no closing tag**Parent** phrasing content | **Children** nothing (void)

An optional line-break opportunity inside a long word or URL.

WHERE IT LIVES IN THE TREE

```
p
├── "example.com/"
├── wbr ◀
├── "very/"
├── wbr ◀
└── "long/path"
```

EXERCISE 1

Put a long URL in a narrow box and add `<wbr>` after each slash. Resize the window.

EXERCISE 2

Do the same with a very long word, e.g. a German compound.

Edits

html > body > ... > changes

Two elements record changes to a document. They can wrap inline text or whole blocks.

<ins>

063

Parent anywhere | **Children** same as its parent allows

Content that was inserted. Use `datetime` and `cite`.

WHERE IT LIVES IN THE TREE

```

article
├── p
│   ├── del "Friday"
│   └── ins datetime="2026-09-20"
│       "Saturday" ◀

```

EXERCISE 1

Show a document revision: old sentence in `<del>`, new sentence in `<ins>` with a `datetime`.

EXERCISE 2

Wrap a whole new paragraph in `<ins>`. Draw the tree showing `<ins>` as the parent of `<p>`.

064

Parent anywhere | **Children** same as its parent allows

Content that was removed.

WHERE IT LIVES IN THE TREE

```

ul
├── li
│   └── del "Buy milk" ◀
└── li "Pay rent"

```

EXERCISE 1

Build a to-do list where completed items are wrapped in `<del>`.

EXERCISE 2

Show a changed meeting date with `<del datetime>` and `<ins datetime>`.

Embedded Content

html > body > ... > media

These elements pull outside content into the tree: images, video, audio, other pages, graphics. Several have a parent-child pattern: the parent picks, the `<source>` children offer options.

`<picture>`

065

Parent phrasing content | **Children** `<source>` elements, then one `<img>`

Offers multiple image sources; the browser picks the best, falling back to `<img>`.

WHERE IT LIVES IN THE TREE

```
figure
├── picture ◀
│   ├── source type="image/webp"
│   ├── source media="(max-width:600px)"
│   └── img
```

EXERCISE 1

Build `picture > source (webp) + img (jpg)` fallback.

EXERCISE 2

Art direction: show a tight crop on phones (`media="(max-width:600px)"`) and a wide photo otherwise.

`<source>` VOID · no closing tag

066

Parent `<picture>`, `<video>`, `<audio>` | **Children** nothing (void)

One media option for its parent to choose from.

WHERE IT LIVES IN THE TREE

```
video controls
├── source type="video/webm" ◀
├── source type="video/mp4" ◀
└── "Fallback text"
```

EXERCISE 1

Give a `<video>` two sources (webm and mp4) plus fallback text.

EXERCISE 2

Give an `<audio>` two sources and a download link as fallback.

`<img>` VOID · no closing tag

067

Parent phrasing content | **Children** nothing (void)

An image. `alt` is required: describe it, or use `alt=""` if decorative.

WHERE IT LIVES IN THE TREE

```
figure
├── img src alt width height ◀
└── figcaption
```

EXERCISE 1

Build a gallery of four images, each with meaningful `alt`, `width` and `height`.

EXERCISE 2

Make one responsive image with `srcset` and `sizes`, and one decorative image with `alt=""`.

`<iframe>`

068

Parent phrasing content | **Children** nothing meaningful

Embeds another whole HTML page (its own separate tree).

WHERE IT LIVES IN THE TREE

```
section
├── h2 "Find us"
├── iframe title="Map" ◀
│   └── (separate document)
```

EXERCISE 1

Embed a map or video with `<iframe>` and a descriptive `title`.

EXERCISE 2

Embed one of your own pages with `sandbox`. Open DevTools and find the second `html` root inside.

<embed> VOID · no closing tag

069

Parent phrasing content | **Children** nothing (void)

Embeds external content handled by the browser or a plugin, e.g. a PDF.

WHERE IT LIVES IN THE TREE

```
section
└─ embed src="notes.pdf" type="..." ◀
```

EXERCISE 1

Embed a PDF with `<embed>`.

EXERCISE 2

Embed the same PDF with `<embed>`, `<iframe>` and `<object>`. Compare the three.

<object>

070

Parent phrasing content | **Children** fallback content

Embeds an external resource, with fallback content as children.

WHERE IT LIVES IN THE TREE

```
object data="chart.svg" ◀
└─ img src="chart.png"
```

EXERCISE 1

Embed an SVG file with `<object>`, with an `<img>` fallback inside.

EXERCISE 2

Build nested fallbacks: `object > object > p`. Explain the order the browser tries.

<video>

071

Parent flow/phrasing content | **Children** `<source>`, `<track>`, fallback

A video player.

WHERE IT LIVES IN THE TREE

```
figure
├─ video controls poster ◀
│   ├── source
│   └─ track kind="subtitles"
└─ figcaption
```

EXERCISE 1

Build `<video controls poster>` with two sources and a fallback download link.

EXERCISE 2

Add a `<track>` of subtitles to your video.

<audio>

072

Parent flow/phrasing content | **Children** `<source>`, `<track>`, fallback

An audio player.

WHERE IT LIVES IN THE TREE

```
figure
├─ figcaption "Episode 1"
└─ audio controls ◀
    └─ source
```

EXERCISE 1

Build a podcast episode: `figure > figcaption + audio controls`.

EXERCISE 2

Build a playlist: an `<ol>` of three `<li>`s, each with a title and an `<audio>`.

<track> VOID · no closing tag

073

Parent <video>, <audio> | **Children** nothing (void)

Timed text tracks: subtitles, captions, chapters (WebVTT files).

WHERE IT LIVES IN THE TREE

```
video
├── source
├── track kind="subtitles" srclang="en" ◀
└── track kind="subtitles" srclang="sw" ◀
```

EXERCISE 1

Add English and Swahili subtitle tracks to a video.

EXERCISE 2

Write a small .vtt file with three cues and connect it as kind="chapters".

<map>

074

Parent phrasing content | **Children** <area> elements

An image map: clickable regions on an image.

WHERE IT LIVES IN THE TREE

```
body
├── img usemap="#floor"
└── map name="floor" ◀
    ├── area
    └── area
```

EXERCISE 1

Make an image with usemap="#m" and a <map> with three clickable areas.

EXERCISE 2

Map a floor plan: one rect room, one circle fountain, one poly garden.

<area> VOID · no closing tag

075

Parent <map> | **Children** nothing (void)

One clickable region inside an image map. Needs alt.

WHERE IT LIVES IN THE TREE

```
map
├── area shape="rect" href alt ◀
└── area shape="circle" href alt ◀
```

EXERCISE 1

Add a rect area that links to a page, with an alt.

EXERCISE 2

Use all three shapes (rect, circle, poly) on one image.

<svg>

076

Parent phrasing content | **Children** SVG elements (its own vocabulary)

Inline vector graphics. Starts a new branch using SVG's own elements.

WHERE IT LIVES IN THE TREE

```
button aria-label="Close"
├── svg viewBox="0 0 24 24" ◀
│   ├── line
│   └── line
```

EXERCISE 1

Draw a circle, rectangle and text with inline <svg>.

EXERCISE 2

Put an SVG "X" icon inside a <button> with an aria-label.

<math>

077

Parent phrasing content | **Children** MathML elements

Inline mathematics (MathML). Starts a branch using MathML's own elements.

WHERE IT LIVES IN THE TREE

```
p
├── math ◀
│   └── msup
│       ├── mi "x"
│       └── mn "2"
```

EXERCISE 1

Write x squared using math > msup > mi + mn.

EXERCISE 2

Write the fraction a/b using <mfraction>. Draw the subtree.

Tables

html > body > ... > table

A table is the strictest subtree in HTML: every element has a fixed parent. Learn the shape once: table > (caption, colgroup, thead, tbody, tfoot) > tr > (th, td). Tables are for data, never layout.

<table>

078

Parent flow content | **Children** caption, colgroup, thead, tbody, tfoot, tr

Tabular data in rows and columns.

WHERE IT LIVES IN THE TREE

```
table ◀
├── caption
├── thead
│   └── tr
├── tbody
│   └── tr
└── tfoot
    └── tr
```

EXERCISE 1

Build a class timetable: 5 days × 4 periods.

EXERCISE 2

Build a student marks table with <thead>, <tbody> and a <tfoot> average row.

<caption>

079

Parent <table> (first child) | **Children** flow content

The table's title.

WHERE IT LIVES IN THE TREE

```
table
├── caption "Term 1 Results" ◀
├── thead
└── tbody
```

EXERCISE 1

Add a <caption> to your timetable.

EXERCISE 2

Move the <caption> to the end of the table and validate. Read the error and explain it.

<colgroup>

080

Parent <table> (after caption) | **Children** <col> elements

Groups columns so they can be styled together.

WHERE IT LIVES IN THE TREE

```
table
├── caption
├── colgroup ◀
│   ├── col span="2"
│   └── col
└── thead
```

EXERCISE 1

Use <colgroup> with span="2" to shade the first two columns.

EXERCISE 2

Use <colgroup> > col to highlight the weekend columns of a calendar table.

<col> VOID · no closing tag

081

Parent <colgroup> | **Children** nothing (void)

One column (or `span` columns) in a colgroup.

WHERE IT LIVES IN THE TREE

```
colgroup
├── col ◀
├── col class="today" ◀
└── col ◀
```

EXERCISE 1

Set different widths for three columns with <col>.

EXERCISE 2

Highlight a single "today" column with <col class>.

<thead>

082

Parent <table> | **Children** <tr>

The header row group.

WHERE IT LIVES IN THE TREE

```
table
├── thead ◀
│   └── tr
│       ├── th
│       └── th
```

EXERCISE 1

Create a <thead> with one header row for a price list.

EXERCISE 2

Create a <thead> with two rows: a group heading spanning columns, then column headings.

<tbody>

083

Parent <table> | **Children** <tr>

The body row group. A table can have several.

WHERE IT LIVES IN THE TREE

```
table
├── thead
├── tbody "Form 1" ◀
└── tbody "Form 2" ◀
```

EXERCISE 1

Put five data rows in a <tbody>.

EXERCISE 2

Use two <tbody>s to split students by class. Style the gap between them.

<tfoot>

084

Parent <table> | **Children** <tr>

The footer row group: totals and summaries.

WHERE IT LIVES IN THE TREE

```
table
├── thead
├── tbody
└── tfoot ◀
    ├── tr
    │   ├── th "Total"
    │   └── td
```

EXERCISE 1

Add a totals row in <tfoot> to an expense table.

EXERCISE 2

Add a <tfoot> with a note spanning all columns via `colspan`.

<tr>

085

Parent <thead>, <tbody>, <tfoot>, <table> | **Children** <th>, <td>

A table row.

WHERE IT LIVES IN THE TREE

```
tbody
├── tr ◀
│   ├── th scope="row"
│   ├── td
│   └── td
```

EXERCISE 1

Build a 3×3 grid of rows and cells, then draw it as a tree.

EXERCISE 2

Build rows where the first cell is a row header <th scope="row">.

<th>

086

Parent <tr> | **Children** flow content (no header/footer/sectioning)

A header cell. Use [scope](#) to say whether it heads a row or column.

WHERE IT LIVES IN THE TREE

```
thead
├── tr
│   ├── th scope="col" "Name" ◀
│   └── th scope="col" "Age" ◀
```

EXERCISE 1

Build a table with both column headers and row headers, using [scope](#) on each.

EXERCISE 2

Build a complex table where data cells reference headers by [id](#) using the [headers](#) attribute.

<td>

087

Parent <tr> | **Children** flow content

A data cell. [colspan](#) and [rowspan](#) merge cells.

WHERE IT LIVES IN THE TREE

```
tr
├── th
├── td colspan="2" ◀
└── td rowspan="2" ◀
```

EXERCISE 1

Build a timetable where lunch spans all columns ([colspan](#)).

EXERCISE 2

Build a schedule where a 2-hour lesson spans two rows ([rowspan](#)). Draw the grid first.

Forms

html > body > ... > form

A form is a subtree that collects user input and sends it somewhere. Every control needs a `<label>`, and every control that sends data needs a `name`.

`<form>`

088

Parent flow content (no nested forms) | **Children** controls, labels, fieldsets, buttons

Collects input and submits it to `action` using `method`.

WHERE IT LIVES IN THE TREE

```
main
├── form action="/login" method="post" ◀
│   ├── label
│   ├── input
│   └── button
```

EXERCISE 1

Build a login form: email, password, submit, `method="post"`.

EXERCISE 2

Build a search form with `method="get"`. Submit it and study the URL.

`<label>`

089

Parent phrasing content | **Children** text + optionally the control

Names a form control. Clicking it focuses the control.

WHERE IT LIVES IN THE TREE

```
form
├── label for="email" ◀
├── input id="email"
├── label ◀
│   └── input type="checkbox"
```

EXERCISE 1

Label one input using `for/id`, and another by wrapping it. Click each label.

EXERCISE 2

Build a checkbox list where clicking any text toggles its box.

`<input>` VOID · no closing tag

090

Parent phrasing content | **Children** nothing (void)

The main form control. Its `type` changes everything: text, email, password, number, date, checkbox, radio, range, file, color...

WHERE IT LIVES IN THE TREE

```
form
├── label
├── input type="email" name required ◀
├── input type="radio" name="size" ◀
└── input type="radio" name="size" ◀
```

EXERCISE 1

Build a registration form using eight different `types`.

EXERCISE 2

Build a radio group (same `name`) and a checkbox group. Explain how their data is sent differently.

<button>

091

Parent phrasing content | **Children** phrasing content (no interactive elements)

A clickable button: `submit`, `reset` or `button`.

WHERE IT LIVES IN THE TREE

```
form
├── input
└── button type="submit" ◀
    ├── svg
    └── "Send"
```

EXERCISE 1

Build a form with three buttons: `submit`, `reset`, `button`. Observe what each does.

EXERCISE 2

Build a button containing an SVG icon and text, and a second submit button using `formaction` to send to a different URL.

<select>

092

Parent phrasing content | **Children** `<option>`, `<optgroup>`

A dropdown list.

WHERE IT LIVES IN THE TREE

```
form
├── label
└── select name="county" ◀
    ├── option
    └── option
```

EXERCISE 1

Build a county/country `<select>` with a placeholder first option.

EXERCISE 2

Build a `<select multiple size="4">` for picking several skills.

<datalist>

093

Parent phrasing content | **Children** `<option>` elements

Suggestions for an `<input>` connected via `list`.

WHERE IT LIVES IN THE TREE

```
form
├── input list="langs"
└── datalist id="langs" ◀
    ├── option value="HTML"
    └── option value="CSS"
```

EXERCISE 1

Give a text input autocomplete suggestions via `<datalist>`.

EXERCISE 2

Give a range slider tick marks using a `<datalist>`.

<optgroup>

094

Parent `<select>` | **Children** `<option>`

Groups options under a label.

WHERE IT LIVES IN THE TREE

```
select
├── optgroup label="Kenya" ◀
│   └── option "Nairobi"
└── optgroup label="Somalia" ◀
    └── option "Mogadishu"
```

EXERCISE 1

Group cities by country with `<optgroup>`.

EXERCISE 2

Add a `disabled` `optgroup`. Test selecting from it.

<option>

095

Parent <select>, <optgroup>, <datalist> | **Children** text

One choice. **value** is sent; the text is shown.

WHERE IT LIVES IN THE TREE

```
select
├─ option value="" "Choose..." ◀
└─ option value="ke" selected "Kenya" ◀
```

EXERCISE 1

Make one option **selected** by default.

EXERCISE 2

Make options whose visible text differs from their **value**. Submit and check what's sent.

<textarea>

096

Parent phrasing content | **Children** text (default value)

Multi-line text input.

WHERE IT LIVES IN THE TREE

```
fieldset
├─ legend
├─ label
└─ textarea name="msg" rows="5" ◀
```

EXERCISE 1

Build a feedback box with **rows**, **maxlength** and a **placeholder**.

EXERCISE 2

Put a <textarea> inside a <fieldset> with a <legend> and a <label>.

<output>

097

Parent phrasing content | **Children** phrasing content

The result of a calculation or user action.

WHERE IT LIVES IN THE TREE

```
form oninput
├─ input name="a"
├─ "+"
├─ input name="b"
└─ output name="sum" ◀
```

EXERCISE 1

Build a two-number adder that updates an <output> live.

EXERCISE 2

Show a range slider's current value in an <output>.

<progress>

098

Parent phrasing content | **Children** fallback text

Completion of a task.

WHERE IT LIVES IN THE TREE

```
section
├─ label "Uploading"
└─ progress value="70" max="100" ◀
```

EXERCISE 1

Show an upload at 70% with <progress value max>.

EXERCISE 2

Show an indeterminate progress bar (no **value**) and explain when to use it.

<meter>

099

Parent phrasing content | **Children** fallback text

A measurement within a known range (not task progress).

WHERE IT LIVES IN THE TREE

```
dl
├── dt "Storage"
└── dd
    └── meter value="0.8" low high optimum
```

EXERCISE 1

Show disk usage with **low**, **high** and **optimum**.

EXERCISE 2

Show password strength with a **<meter>** beneath a password input.

<fieldset>

100

Parent flow content | **Children** **<legend>** first, then controls

Groups related controls.

WHERE IT LIVES IN THE TREE

```
form
├── fieldset
│   ├── legend "Shipping"
│   └── input
└── fieldset disabled
    └── legend "Billing"
```

EXERCISE 1

Build a checkout with Shipping and Billing address fieldsets.

EXERCISE 2

Make the Billing fieldset **disabled** and note every control inside is disabled.

<legend>

101

Parent **<fieldset>** (first child) | **Children** phrasing content, headings

The caption for a fieldset.

WHERE IT LIVES IN THE TREE

```
fieldset
├── legend "T-shirt size"
├── input type="radio"
└── input type="radio"
```

EXERCISE 1

Wrap a radio group in a fieldset with a **<legend>** asking the question.

EXERCISE 2

Nest one fieldset inside another, each with its own **<legend>**. Draw the tree.

Interactive Elements

html > body > ... > widgets

Built-in widgets that work without JavaScript (or with very little).

<details>

102

Parent flow content | **Children** <summary> first, then content

A disclosure widget: click to show or hide.

WHERE IT LIVES IN THE TREE

```

section
├── h2 "FAQ"
├── details name="faq" ◀
│   ├── summary
│   └── p
└── details name="faq" ◀
  
```

EXERCISE 1

Build an FAQ with three <details> items.

EXERCISE 2

Give all three the same `name` so only one opens at a time (an accordion).

<summary>

103

Parent <details> (first child) | **Children** phrasing content or one heading

The always-visible label of a <details>.

WHERE IT LIVES IN THE TREE

```

details open
├── summary ◀
│   └── h3 "Is it free?"
└── p
  
```

EXERCISE 1

Put an <h3> inside a <summary> and check the heading appears in the outline.

EXERCISE 2

Use the `open` attribute to make one item start expanded.

<dialog>

104

Parent flow content | **Children** flow content

A dialog box or modal window.

WHERE IT LIVES IN THE TREE

```

body
├── button "Open"
└── dialog ◀
    └── form method="dialog"
        ├── p
        └── button "Close"
  
```

EXERCISE 1

Build a modal opened with `showModal()` from a button click.

EXERCISE 2

Put a <form `method="dialog"`> inside the dialog so its button closes it without JavaScript.

Scripting and Components

html > head/body > scripts

These elements connect HTML to JavaScript: running scripts, drawing pixels, and building reusable components.

<script>

105

Parent <head> or <body> | **Children** JavaScript (or a src)

Runs JavaScript. External scripts use src; defer waits for the tree to be built.

WHERE IT LIVES IN THE TREE

```
html
├── head
│   └── script src="app.js" defer ◀
└── body
    └── p id="out"
```

EXERCISE 1

Write an inline <script> at the end of <body> that changes the text of a <p>.

EXERCISE 2

Move the script to <head> without defer — it breaks. Add defer and explain why it works.

<noscript>

106

Parent <head> or <body> | **Children** fallback content

Content shown only when JavaScript is off.

WHERE IT LIVES IN THE TREE

```
body
├── noscript ◀
│   └── p "Enable JavaScript"
└── div id="app"
```

EXERCISE 1

Add a <noscript> message and test by disabling JavaScript in DevTools.

EXERCISE 2

Use <noscript> to offer a plain link as fallback for a JS-powered button.

<template>

107

Parent anywhere | **Children** any content (inert)

A hidden fragment of HTML that JavaScript can clone. Not rendered.

WHERE IT LIVES IN THE TREE

```
body
├── ul id="list"
└── template id="card" ◀
    └── li
        ├── h3
        └── p
```

EXERCISE 1

Define a card <template> and clone it three times with JavaScript.

EXERCISE 2

Use a <template> of a <tr> to add rows to a table.

<slot>

108

Parent inside a shadow tree | **Children** fallback content

A placeholder inside a web component, filled by the component's children.

WHERE IT LIVES IN THE TREE

```
my-card
├── #shadow-root
│   ├── h2
│   │   └── slot name="title" ◀
│   └── slot ◀
```

EXERCISE 1

Build a `<my-card>` custom element with a shadow root containing one `<slot>`.

EXERCISE 2

Add named slots (`title`, `body`) and fill them from the outside.

<canvas>

109

Parent phrasing content | **Children** fallback content

A pixel drawing surface controlled by JavaScript.

WHERE IT LIVES IN THE TREE

```
figure
├── canvas width="300" height="150" ◀
│   └── p "Chart of sales"
└── figcaption
```

EXERCISE 1

Draw a filled rectangle and a circle on a `<canvas>` with JavaScript.

EXERCISE 2

Put meaningful fallback content inside the canvas describing what it shows.

Dead Branches

obsolete elements – recognise, never write

You'll meet these in old code and old tutorials. Browsers may still render some of them, but they are not part of modern HTML. When you see one, replace it.

<acronym>	use abbr	<strike>	use s or del
<applet>	use object/embed	<tt>	use code/kbd/samp
<basefont>	use CSS	<param>	use object attributes
<big>	use CSS	<nobr>	use CSS white-space
<blink>	use CSS	<xmp>	use pre
<center>	use CSS	<plaintext>	use pre
<dir>	use ul	<listing>	use pre
	use CSS	<rb>	use ruby base text
<frame>	use iframe	<rte>	use rt
<frameset>	use CSS layout	<keygen>	removed
<noframes>	—	<menuitem>	removed
<marquee>	use CSS animation	<spacer>	use CSS

Final challenge

Build a one-page school website that uses at least one element from every branch (1–10). Before writing any HTML, draw the full tree on one sheet of paper. Then build it, open DevTools, and check that every node sits exactly where you drew it. Validate. Any difference between your drawing and the browser's tree is a rule you haven't learned yet.

Reference: MDN Web Docs — HTML elements reference (developer.mozilla.org) · WHATWG HTML Living Standard (html.spec.whatwg.org)